

# Capítulo 1:

# Introducción a las Estructuras de Datos

Laura Viviana Galvis

Escuela de Ingeniería de Sistemas e Informática  
Facultad de Ingenierías Fisicomecánicas  
Universidad Industrial de Santander

2025-2

Escuela de  
Ingeniería de  
Sistemas e  
Informática

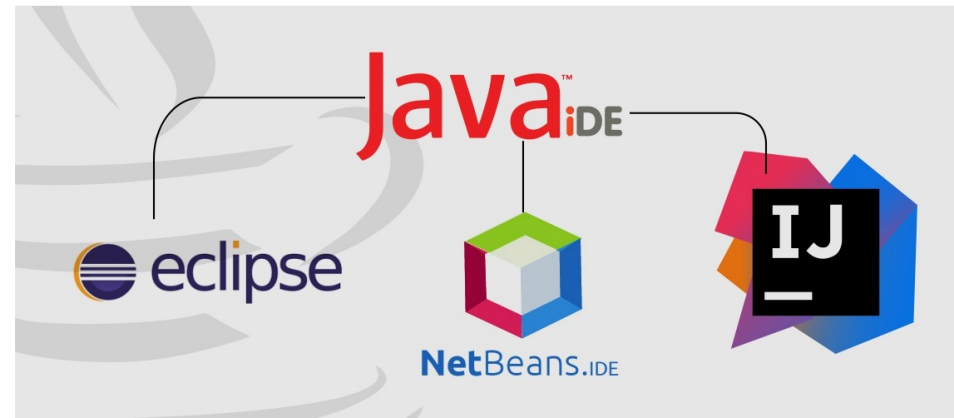
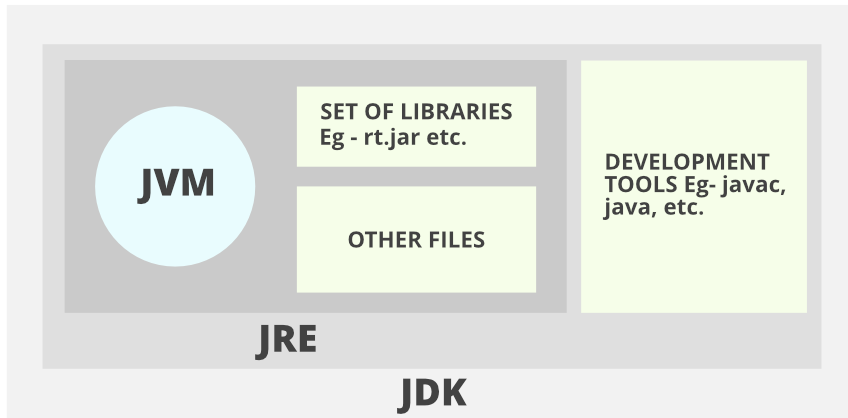


Universidad  
Industrial de  
Santander

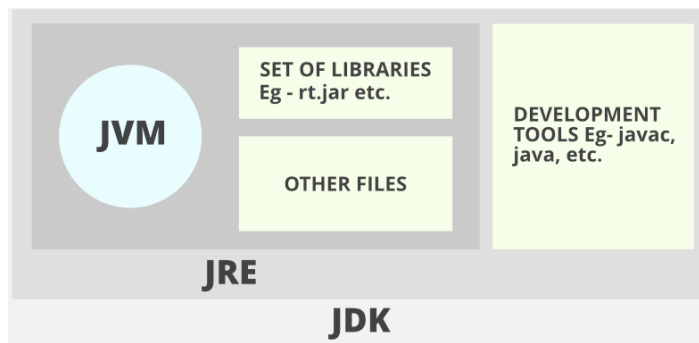


# Agenda

- ¿Qué necesitamos instalar para poder programar en Java?
- ¿Qué es un tipo abstracto de datos (TAD)?
- ¿Qué tipos de TAD existen?



# JDK, JRE y JVM

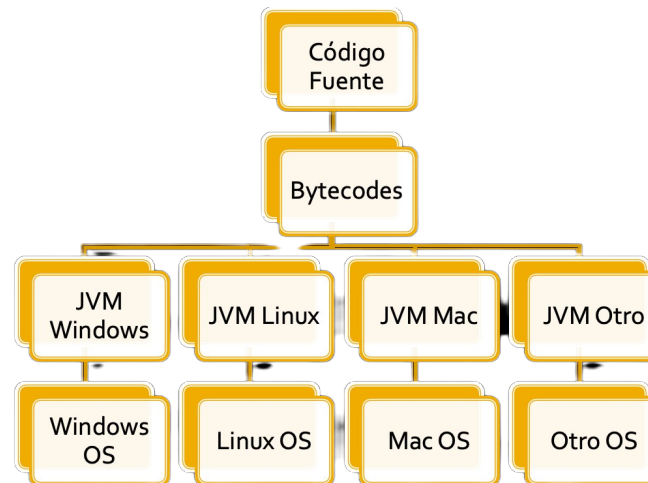


## JDK (Java Development Kit)

- Solamente para desarrolladores
- Contiene:
  - Herramientas de desarrollo
  - Ambiente de ejecución
  - API Java SE (compilada y código fuente)
  - Programas de ejemplo y documentación

## JRE (Java Runtime Environment)

- Necesaria para la ejecución de programas JAVA
- La única que los clientes deben instalar



## Herramientas disponibles con JDK

- Compilador (*javac*); Intérprete (*java*)
- Generador de documentación (*javadoc*)
- Depurador (*jdb*); Generador de paquetes (*jar*)

1. Codificación (Crear archivo Saludo.java)

```
public class Saludo{  
    public static void main(String [] args){  
        System.out.println("Bienvenido a Java");  
    }  
}
```

2. Compilación (javac Saludo.java)

3. Ejecución (java Saludo)

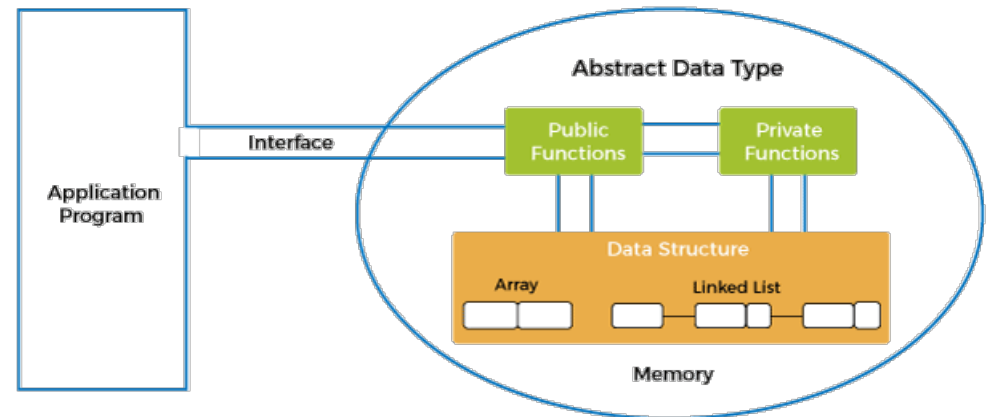
## IDE (Integrated Development Environment)



# Contenidos del capítulo

1. Tipos de datos abstractos (TDA o ADT)
2. Modularidad
3. Uso de TDA
4. Memoria estática y dinámica

Modelo de tipo de datos abstracto



# Estructura de datos

Colección de **datos** que pueden ser caracterizados por su organización y las **operaciones** que se definen en ella.

Tipos de datos más frecuentes en lenguajes de programación:

## Datos Simples o Primitivos

*estándar*

entero (*integer*)  
real (*real*)  
carácter (*char*)  
lógico (*boolean*)

*definido por el programador  
(no estándar)*

subrango (*subrange*)  
enumerativo (*enumerated*)

Propios de Pascal

## Datos Compuestos o Estructurados

*estáticos*

arrays (*vectores/matrices*)  
registros (*record*)  
ficheros (*archivos*)  
conjuntos (*set*)  
cadenas (*string*)

Propio de Pascal

*dinámicos*

listas (*pilas/colas*)  
listas enlazadas  
árboles  
grafos

# Estructura de datos en Java

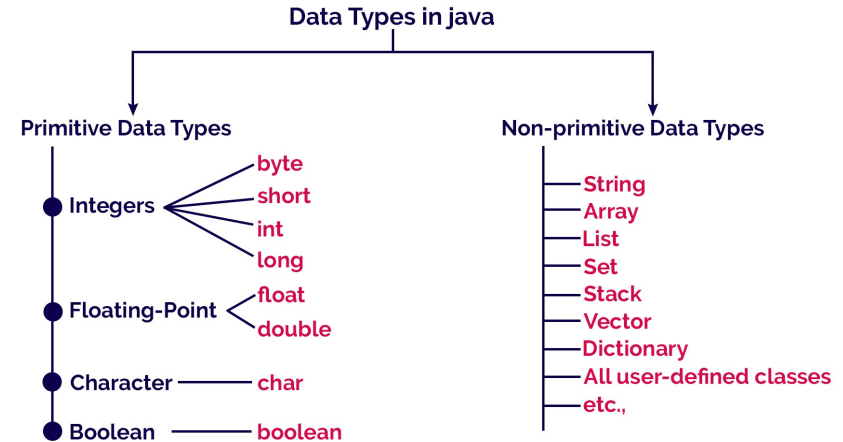
El lenguaje de programación Java tiene un gran conjunto de datos.

Clasificación:

- **Primitivos:** Valores únicos sin capacidades especiales
- **No primitivos:** Tipos de datos creados por el usuario

Ejemplo tipo de dato **“int”**:

- **Dominio de valores:** ..., -2, -1, 0, 1, 2, ...
- **Operaciones:** suma, resta, multiplicación, módulo, ¿división?



www.btechsmartclass.com

# Tipos de Datos Primitivos

| TYPE    | DESCRIPTION             | DEFAULT | SIZE    | EXAMPLE LITERALS                                 | RANGE OF VALUES                                         |
|---------|-------------------------|---------|---------|--------------------------------------------------|---------------------------------------------------------|
| boolean | true or false           | false   | 1 bit   | true, false                                      | true, false                                             |
| byte    | twos complement integer | 0       | 8 bits  | (none)                                           | -128 to 127                                             |
| char    | unicode character       | \u0000  | 16 bits | 'a', '\u0041', '\t01', '\w', '\v', '\n', '\beta' | character representation of ASCII values 0 to 255       |
| short   | twos complement integer | 0       | 16 bits | (none)                                           | -32,768 to 32,767                                       |
| int     | twos complement integer | 0       | 32 bits | -2, -1, 0, 1, 2                                  | -2,147,483,648 to 2,147,483,647                         |
| long    | twos complement integer | 0       | 64 bits | -2L, -1L, 0L, 1L, 2L                             | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| float   | IEEE 754 floating point | 0.0     | 32 bits | 1.23e100f, -1.23e-100f, .3f, 3.14F               | upto 7 decimal digits                                   |
| double  | IEEE 754 floating point | 0.0     | 64 bits | 1.23456e300d, -1.23456e-300d, 1e1d               | upto 16 decimal digits                                  |

# 1. Tipos de Datos Abstractos (TDA)

Un TDA es un tipo de datos que define operaciones sobre valores usando funciones, sin especificar qué se encuentra dentro de cada función, ni cómo las operaciones se ejecutan.

¿Recuerdan los pilares de la POO?

¿Particularmente, qué recuerda sobre **abstracción**?





# Ejemplo de Abstracción (Celular)

Especificaciones de un teléfono móvil:

- 4 GB RAM
- Procesador Snapdragon 2.2 GHz
- Pantalla LCD de 5 pulgadas
- Doble cámara
- OS: Android 8.0

Acciones que puede hacer un teléfono móvil:

- **Llamar**
- **Enviar mensaje**
- **Tomar foto**
- **Tomar video**



| Celular                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- memoriaRam</li> <li>- nombreProcesador</li> <li>- tamañoPantalla</li> <li>- numeroDeCamaras</li> </ul> |
| <ul style="list-style-type: none"> <li>+ llamar()</li> <li>+ enviarMensaje()</li> <li>+ tomarFoto()</li> <li>+ tomarVideo()</li> </ul>          |

Atributos + Métodos (Comportamientos)

**Clase:** Unidad Fundamental Java

## Implementación

```

1. class Smartphone
2. {
3.     private:
4.     int ramSize;
5.     String processorName;
6.     float screenSize;
7.     int cameraCount;
8.     String androidVersion;
9.     public:
10.    void llamar();
11.    void enviarMensaje();
12.    void tomarFoto();
13.    void tomarVideo();
14. }
```

# Ejemplo de TDA (Vehículo)

Especificaciones de un vehículo:

- Carro
- Chevrolet Corsa
- Modelo 2020
- 4 ruedas, 5 pasajeros
- 2 toneladas
- Motor de 1400cc a gasolina
- Transmisión mecánica

Acciones que puede hacer un vehículo:

- **acelerar():** Podemos acelerar
- **frenar():** Podemos frenar
- **cambiarMarcha():** Podemos hacer cambios
- **girar():** Podemos arrancar el vehiculo



| Vehiculo                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- tipoVehiculo</li> <li>- marca</li> <li>- modelo</li> <li>- numeroRuedas</li> <li>- peso</li> <li>- tipoTransmision</li> </ul> |
| <ul style="list-style-type: none"> <li>+ acelerar()</li> <li>+ frenar()</li> <li>+ cambiarMarcha()</li> <li>+ girar()</li> </ul>                                       |

## Implementación en Java

```

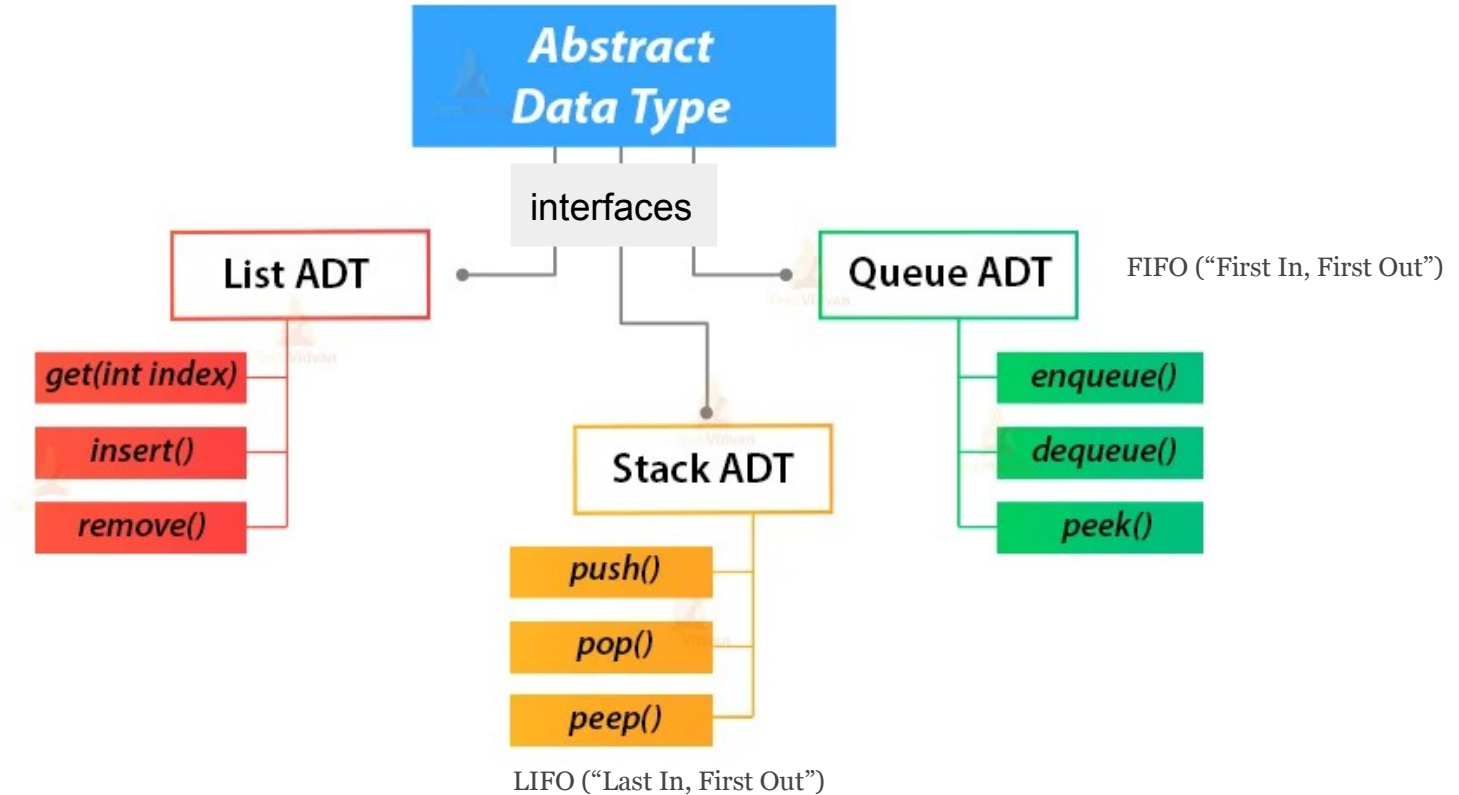
1. class Car
2. {
3.     private:
4.         int numeroRuedas;
5.         int numeroAsientos;
6.         String color;
7.         String marca;
8.
9.     public:
10.        void frenar();
11.        void acelerar();
12.        void cambiarMarcha();
13.        void girar();
14. }
```

# Entonces, ¿qué es un TDA?

Consta de 2 partes:

1. La descripción matemática del conjunto de datos.
  2. La descripción de las operaciones definidas en ciertos elementos de ese conjunto de datos.
- 
- En Java, un TDA se implementa por medio de clases
  - Recuerde: Las clases contienen **atributos** y **métodos**

# Various Java Abstract Data Types



**Nota:** Se definen las funciones, pero **NO** se especifica cómo se implementan

# Ventajas de usar TDA

Si usamos un TDA “Pila” en algún momento de nuestro diseño:

- El usuario puede utilizar los métodos **pop()** y **push()** para remover o agregar elementos en la pila
- Las pilas (como lo vamos a ver más adelante) se pueden implementar usando arrays o listas enlazadas
- Sin embargo, el usuario final NO sabe qué tipo de implementación se utilizó
- Incluso, si en el futuro aparece una manera más eficiente de hacer los métodos pop() y push(), se actualizará la implementación sin que el usuario lo note (**será transparente**)

## 2. Modularidad

# ¿Qué es la Modularidad?

En POO, la lógica de programación se realiza en pequeños trozos (**módulos/métodos**)

Los métodos definen la funcionalidad o comportamiento de un objeto

## ¿Para qué sirven?

1. Reutilización de código (no debes re-escribir la funcionalidad cada vez que la necesites)
2. Mantienen el orden y ayudan a mejorar la comprensión del código
3. Permite la separación de tareas entre los programadores (cada uno implementa ...)
4. Ayuda a resolver problemas complejos al dividir funcionalidad en pequeños módulos

# ¿Qué contiene un método?

- Un bloque de código con su respectivo nombre
  - El nombre describe una acción o comportamiento
  - Se escribe convencionalmente como verbo infinitivo
- El método puede recibir parámetros de entrada
- El método puede retornar un tipo de dato conocido
- Describe cómo se van a realizar las acciones

## Sintaxis:

```
tipoDatoRetorno nombre_del_metodo(parametros) {  
    Acciones a realizar;  
}
```

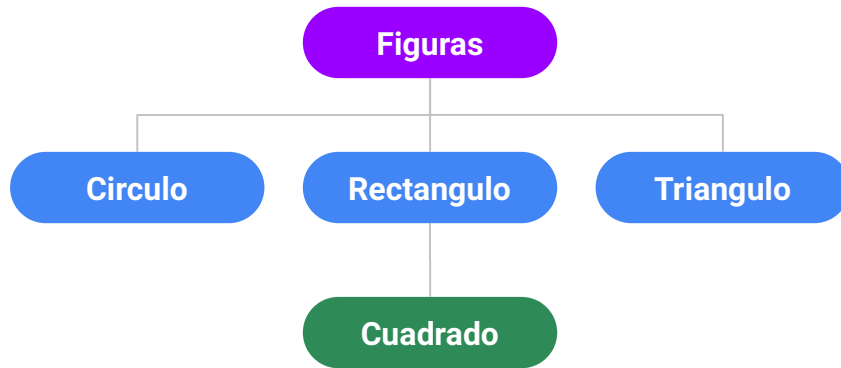
```
int sumar (int x, int y){  
    return (x+y);  
}
```

```
void saludar (){  
    System.out.println("Hola");  
}
```

```
float calcularAreaCirculo(int radio){  
    return (Math.PI * Math.pow(radio,2));  
}
```



# Ejemplo de un TDA en Java (Figuras geométricas)



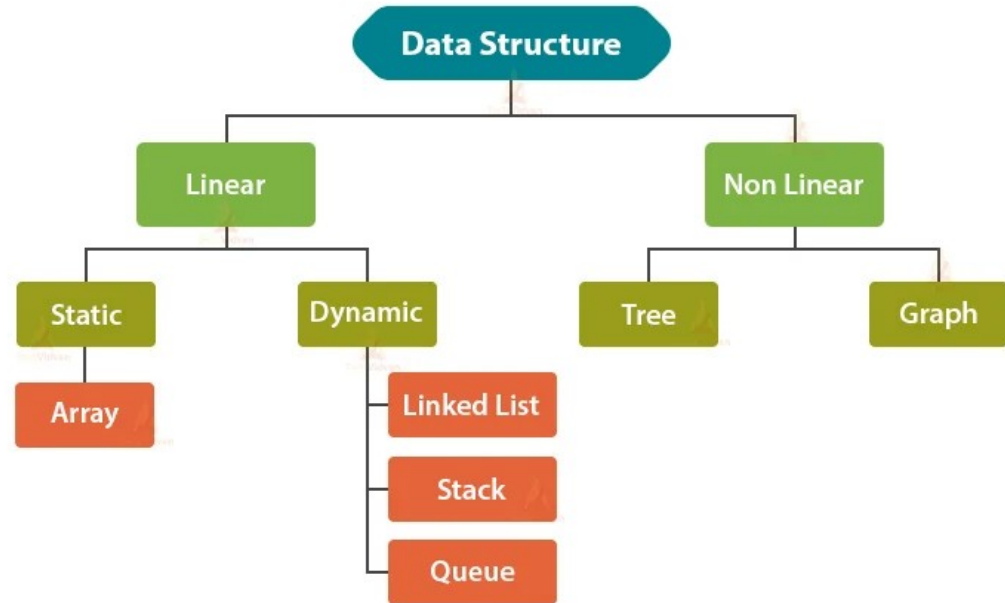
## Ejercicio en clase (para recordar POO):

1. Debemos crear 5 clases: **Figura**, **Circulo**, **Rectangulo**, **Triangulo** y **Cuadrado**
2. **Circulo**, **Rectangulo** y **Triangulo** heredan de la clase **Figura** quien define el punto (x,y) y los métodos abstractos *obtenerArea()* y *obtenerPerimetro()*
3. Similarmente la clase **Cuadrado** hereda de la clase **Rectangulo**.
4. Recordando el concepto de herencia, cada clase hija debe implementar los métodos abstractos de la superclase.
5. El objetivo es implementar las clases, incluyendo los *getters & setters*, y además crear la clase **Main**, donde crearemos una figura de cada clase hija e imprimiremos en consola su área y su perímetro.

### 3. Ejemplos y usos de TDA

# Ejemplos de TDA comunes y Usos

- Listas enlazadas
  - Procesadores de texto
  - Colas y pilas
- Pilas (LIFO)
  - Expresiones aritméticas
- Colas (FIFO)
  - Cajeros
  - Peajes
- Árboles
  - Búsqueda en bases de datos
  - Orden en bases de datos
- Grafos
  - Programación de transporte (camino más corto)
  - Conexiones en redes sociales



# Ejercicio en clase

Análisis de casos y propuesta de TDA

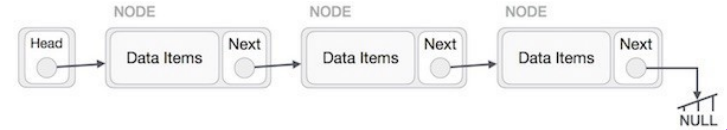
# Listas enlazadas

- Estructura de datos que representa una secuencia de **nodos**, conectados a través de **enlaces**
- Cada nodo está dividido en 2 (simple) o 3 partes (doble)
  - Simple: Donde se almacena el dato, y la dirección del siguiente nodo
  - Doble: Donde se almacena el dato, la dirección del siguiente nodo, y la dirección del anterior nodo
- Los enlaces tienen terminación NULL (excepto circulares)
  - Cuando el puntero al siguiente nodo es NULL, la lista ha llegado al final

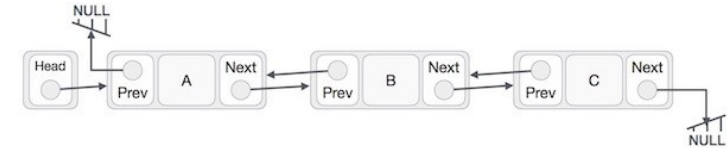
## Aplicaciones:

- Gestión de la lista de canciones en reproductor multimedia.
  - Inicio/final, agregar canción, escuchar anterior/siguiente
- Combinaciones de teclado para *shortcuts*
- Historial de navegación
- Operaciones “hacer”, “deshacer”
- Una lista circular se usa en los videojuegos multijugador para controlar el orden de los jugadores.

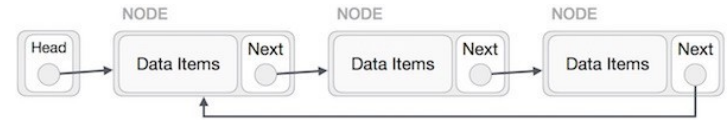
### Listas enlazada simple



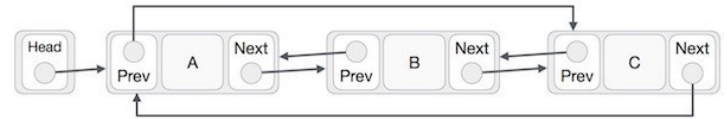
### Listas enlazada doble



### Listas enlazada circular (no tiene fin)



### Listas enlazada circular doble



# Pilas (Last-In First-Out)

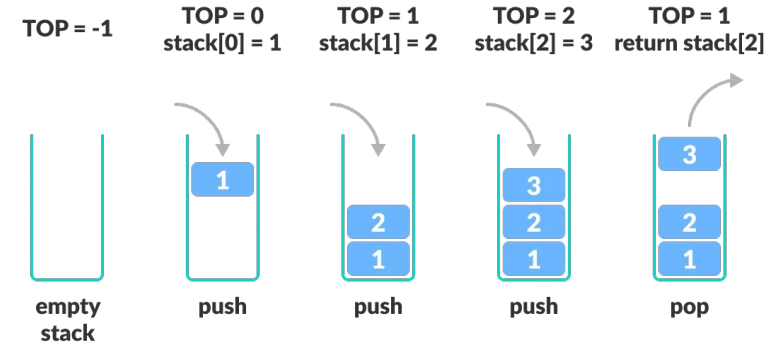
- Es una estructura de datos lineal que sigue el principio de último en entrar primero en salir (LIFO).
- Operaciones permitidas:
  - push()**: agrega un elemento a la parte superior
  - pop()**: elimina un elemento de la parte superior
  - isEmpty()**: comprueba si la pila está vacía
  - isFull()**: comprueba si la pila está llena
  - peek()**: obtiene el valor del elemento superior sin eliminarlo



## Aplicaciones:

- Análisis de sintaxis
- Comprobación de paréntesis en aritmética
  - $2 + 4 / 5 * (7 - 9)$
- Control de funciones o subrutinas activas

```
c[d]      // correct
a{b[c]}e  // correct
a{b[c]d}e // not correct
```



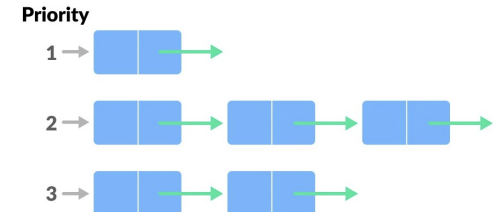
# Colas (First-In First-Out)

- Es una estructura de datos lineal que sigue el principio de primero en entrar primero en salir (FIFO).
- Tipos: simples, circular, de prioridad, y doble
- Operaciones permitidas:
  - **enqueue()**: agrega un elemento al final de la cola
  - **dequeue()**: elimina un elemento del frente de la cola
  - **isEmpty()**: comprueba si la cola está vacía
  - **isFull()**: comprueba si la cola está llena
  - **peek()**: obtiene el valor del frente de la cola sin eliminarlo



## Aplicaciones:

- Programación de tareas en un PC (imprimir, abrir programas, copiar archivos, etc.)
- Call centers (pedir citas, etc.)
- Listas de reproducción (música, videos youtube, etc)
- Envío de mensajes por Whatsapp y no se tiene conexión a internet

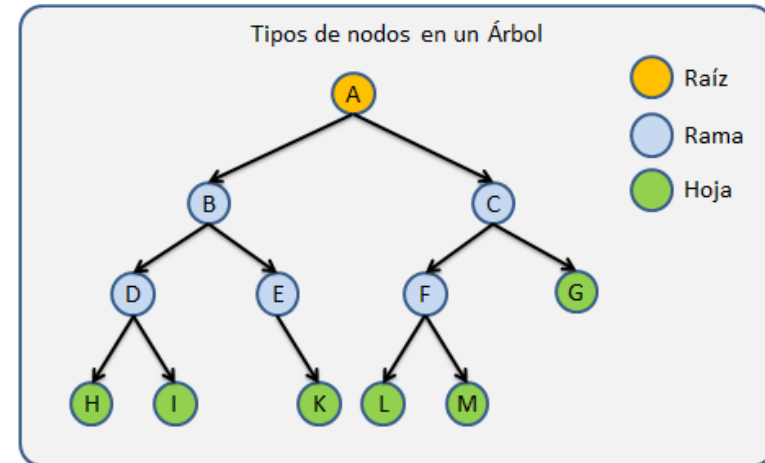


# Árboles

- Un árbol es una estructura de datos jerárquica, no lineal, y dinámica que consta de nodos conectados por aristas.
- Existen 3 tipos de nodos: Raíz, rama, hoja.
- Tipos de árboles: Binarios y multi-camino
- Un árbol no debe tener ciclos
- Operaciones permitidas:
  - Búsqueda, inserción, eliminación

## Aplicaciones:

- Los árboles son un método eficiente para búsquedas grandes y complejas
- Casi todos los sistemas operativos almacenan sus archivos en árboles (carpetas/subcarpetas)
- Indexación en bases de datos





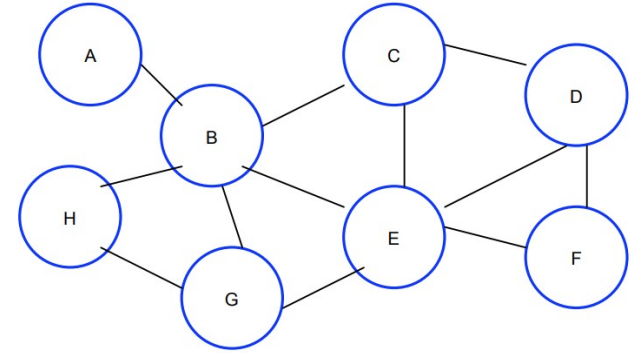
# Grafos

$$G = (V, A)$$

$$V = \{A, B, C, D, E, F, G, H\}$$

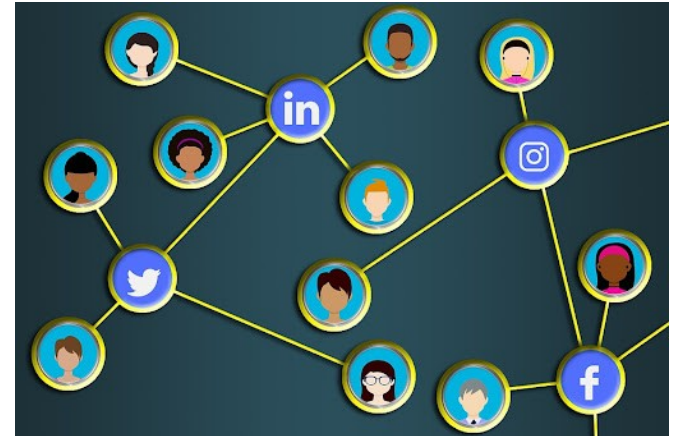
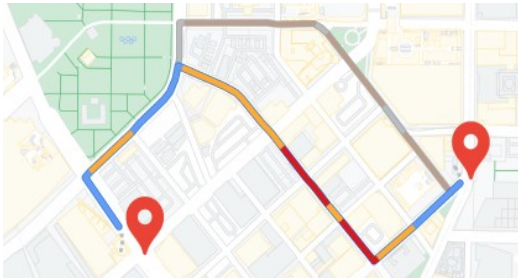
$$A = \{\{A, B\}, \{B, C\}, \{B, E\}, \{B, G\}, \{B, H\}, \{C, D\}, \{C, E\}, \{D, F\}, \{D, E\}, \{E, G\}, \{E, H\}, \{F, G\}\}$$

- Un grafo,  $G$ , es un par, compuesto por dos conjuntos, uno de vértices ( $V$ ) y uno de arcos ( $A$ ).
  - $A$  es un conjunto de pares de vértices, estos pares se conocen con el nombre de arcos o ejes del grafo.
- Se suele utilizar la notación  $G = (V, A)$  para identificar un grafo.
- Tipos de grafos: Dirigidos, no dirigidos, etiquetados, ponderados



## Aplicaciones:

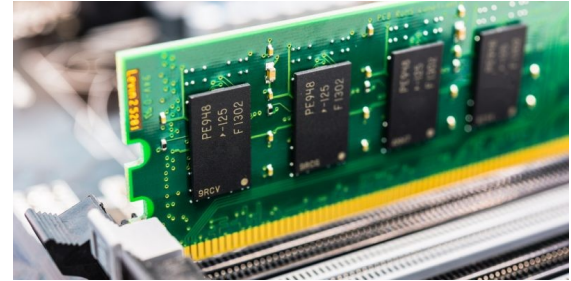
- Representación de redes y rutas en aplicaciones de comunicación, transporte y viajes.
- Interconexiones en redes sociales.



## 4. Memoria estática y dinámica

# Tipos de Memoria

**Memoria:** Espacio lógico (volátil) para almacenar información



**Memoria estática:** Espacio de memoria que se reserva con antelación y NO cambia de tamaño en tiempo de ejecución

```
float alturaEstudiantes[] = new float[25];  
int edadEstudiantes[] = new int[25];
```

**Memoria dinámica:** Espacio de memoria que puede variar en tiempo de ejecución

```
ArrayList<String> nombreEstudiantes;  
nombreEstudiantes = new ArrayList<String>();
```

# Memoria Estática

## Ventajas

- La lógica es fácil
- Rápida (en cuanto a rendimiento)
- Óptima para resolver problemas pequeños y medianos
- Sabremos con anticipación si el programa podrá ejecutarse en nuestra máquina
- Type-safe (only **int** or **String**, but not both)



## Desventajas

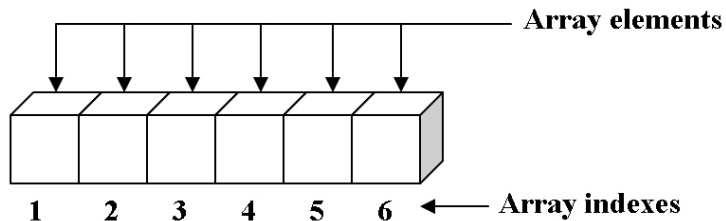
- No es óptima con grandes cantidades de datos
- No puede modificarse en tiempo de ejecución
- Se debe saber con anticipación su tamaño
- Desperdicio de memoria cuando no se utiliza en su totalidad
- Remover elementos es complicado





# Memoria Estática (TDA **Arrays** de Java)

- Clase **Arrays** de Java
- Se debe definir el tamaño de antemano
- Este tamaño es fijo y no se puede modificar
- Contiene manejo de datos y comportamientos
- Algunos métodos importantes:
  - `sort()`
  - `fill()`
  - `equals()`
  - ...



```
import java.util.Arrays;
import java.util.Scanner;

int edadEstudiantes[] = new int[25];
Scanner scan= new Scanner(System.in);

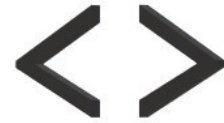
for (int i=0; i<edadEstudiantes.length; i++){
    System.out.print("Digite edad del estudiante " + (i+1) + ": ");
    edadEstudiantes[i] = scan.nextInt(); // Agregar
datos
}

for (int i=0; i<edadEstudiantes.length; i++){
    System.out.println("Edades[" + i + "] = " + edadEstudiantes[i]);
}

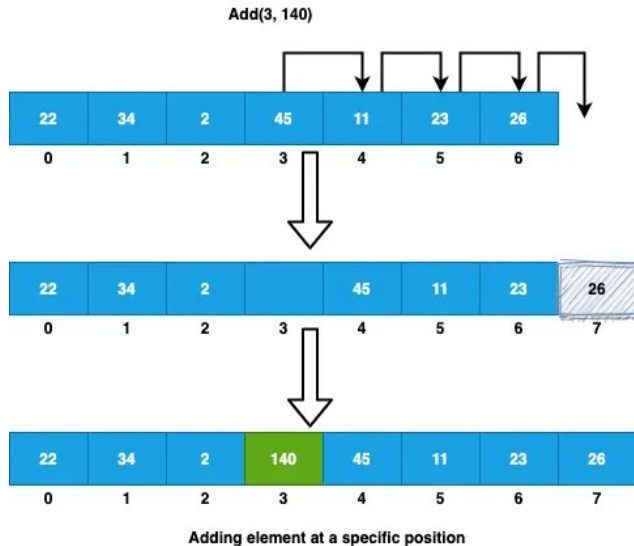
// Usando métodos del TDA Arrays
Arrays.sort(edadEstudiantes);

for (int i=0; i<edadEstudiantes.length; i++){
    System.out.println("Edades[" + i + "] = " + edadEstudiantes[i]);
}
```

# Memoria Dinámica (TDA **ArrayList** de Java)



- Clase **ArrayList** de Java
- Algunos métodos importantes:
  - add(), get(), set(), remove(), size(), ...



```
import java.util.ArrayList;
import java.util.Scanner;

ArrayList<String> nombreEstudiantes = new ArrayList<String>();
String nombre;
Scanner scan = new Scanner(System.in);

do{
    System.out.print("Digite nombre del estudiante: ");
    nombre = scan.nextLine();
    nombreEstudiantes.add(nombre);           // Agregar datos
}while(!nombre.equals("STOP"));

for(int i=0; i<nombreEstudiantes.size()-1; i++){
    System.out.println("Nombre " + i + ": " + nombreEstudiantes.get(i));
}

// Reemplazar un registro
System.out.println("Nombres después de reemplazar:");
nombreEstudiantes.set(2, "Nombre modificado");

for(int i=0; i<nombreEstudiantes.size()-1; i++){
    System.out.println("Nombre " + i + ": " + nombreEstudiantes.get(i));
}

// Eliminar un registro
System.out.println("Nombres después de remover:");
nombreEstudiantes.remove(0);

for(int i=0; i<nombreEstudiantes.size()-1; i++){
    System.out.println("Nombre " + i + ": " + nombreEstudiantes.get(i));
}
```

# Leer para la próxima clase



- ¿Qué entiende por recursividad?
- ¿Qué es la serie de Fibonacci?
- ¿Cómo se calcula el factorial de un número?
- ¿Conoces el juego de las torres de Hanoi?

